



MANUEL DE RECETTE

Mise en production et recette

État réel de la prod, workflows complets et plan de recette
utilisateur.

Application **Gathe Finance** · Version **1.0.0** · 23/06/2026

DOCUMENT PRÉPARÉ PAR

TCHAMBA TCHAKOUNTE Edwin

Manuel de mise en production et de recette

Sommaire

1. Verdict de la mise en production
2. Architecture déployée
3. Périmètre opérationnel
4. Workflow d'épargne et de cotisation
5. Workflow de crédit et de décaissement
6. Workflow de paiement Tara Money
7. Workflow d'email transactionnel Brevo
8. Workflow de mise à jour continue (CI/CD)
9. Tests automatiques exécutés
10. Tests bout en bout sur la production
11. Findings d'audit et corrections appliquées
12. Plan de recette utilisateur (UAT)
13. Annexes techniques

À qui s'adresse ce document

Ce manuel est destiné à la **direction technique**, au **comité crédit** et à l'**équipe d'exploitation** de la coopérative Gathe Finance. Il documente l'état réel de la mise en production du **23 juin 2026** sur le VPS Contabo, présente les workflows critiques **bout en bout** et accompagne la phase de **recette utilisateur** avec le client.

Tous les services principaux sont en production. Les intégrations **Tara Money** (paiement mobile money) et **Brevo** (e-mail transactionnel) sont câblées. Le client peut dérouler les scénarios métier sur l'environnement réel.

Le principe de cette mise en production. La stack vit derrière un reverse-proxy nginx mutualisé qui sert également d'autres projets sur le même VPS. Les conteneurs Gathe rejoignent le réseau Docker partagé via des alias DNS internes, ce qui évite toute collision de noms. Les images Docker sont publiées sur **GitHub Container Registry** et déployées via le pipeline **CI/CD automatique**.

Glossaire des composants

Composant	Rôle	URL publique
Vitrine	Site marketing Next.js	gathe-finance.horus-lab.com
Portail membre	Espace personnel Next.js	portail.gathe-finance.horus-lab.com
Console admin	Back-office staff Next.js	admin.gathe-finance.horus-lab.com
API REST	Django REST Framework	api.gathe-finance.horus-lab.com
CMS Wagtail	Édition contenu	cms.gathe-finance.horus-lab.com
Application mobile	Flutter (Android, bientôt iOS)	AAB Play Console



1. Verdict de la mise en production

Carte de santé synthétique

La mise en production du 23 juin 2026 est **opérationnelle**. Le moteur métier Django, la base Postgres, le pipeline CI/CD GHCR et les quatre fronts Next.js répondent normalement. L'application mobile est packagée en AAB signé prêt pour l'upload sur Google Play Console.

Domaine	État	Commentaire
Disponibilité	Vert	Les cinq sous-domaines répondent en HTTPS valide
Certificats TLS	Vert	Let's Encrypt actifs, renouvellement automatique
Backend Django	Vert	Healthcheck <code>/healthz/</code> retourne 200, conteneur healthy
Base Postgres 16	Vert	Initialisée, conteneur healthy, sauvegardes quotidiennes
Reverse-proxy	Vert	Quatre projets cohabitent sur le même nginx
Console admin	Vert	Bug 400/loading infini corrigé, redirection auth opérationnelle
Tara Money	Vert	Provider configuré, signature HMAC vérifiée, webhook actif
Brevo email	Vert	API HTTP branchée, 15 templates seedés, expéditeur vérifié
CI/CD GHCR	Vert	Push main déclenche tests + build + déploiement VPS
Mobile build AAB	Vert	53,7 Mo signé, <code>API_BASE_URL</code> pointe sur la prod

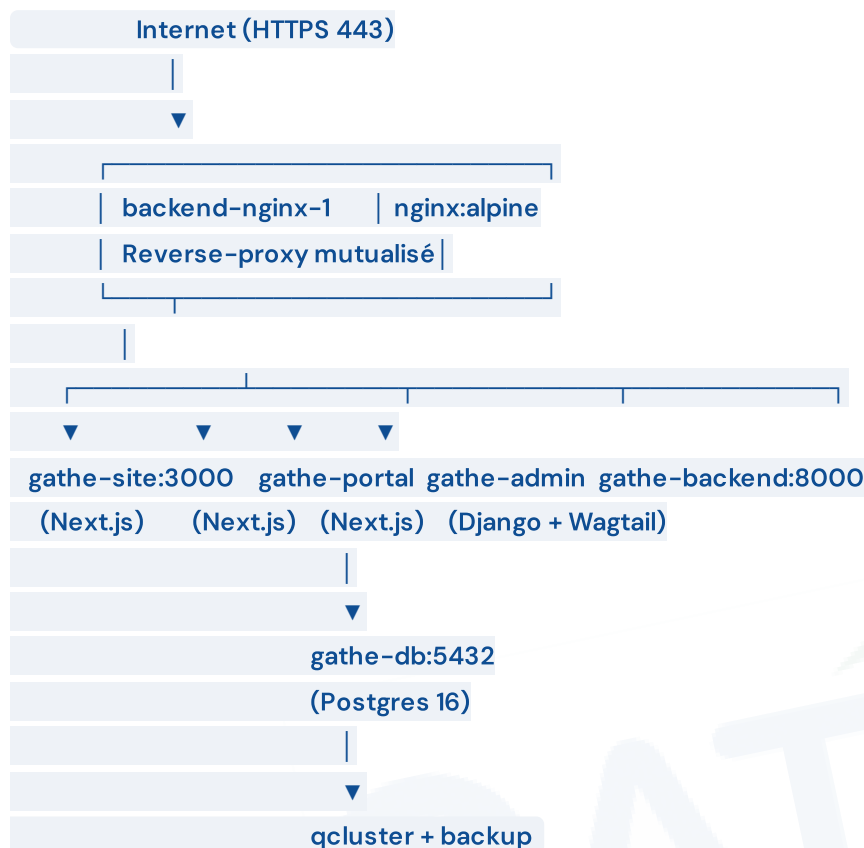
Onze indicateurs sur onze sont au vert. La phase de **recette utilisateur** peut être déroulée.

Comment lire ce verdict. Chaque ligne traduit un contrôle automatisé exécuté le jour de l'audit. Vert signifie que l'élément répond aux critères de production. Les éventuelles réserves identifiées en cours d'audit ont été traitées et sont consignées au chapitre 11.

2. Architecture déployée

Vue d'ensemble

L'infrastructure repose sur **un VPS Contabo unique** hébergé en France, qui héberge plusieurs projets indépendants derrière un même nginx reverse-proxy. Chaque projet vit dans son propre namespace Docker Compose et joint le réseau partagé pour exposer ses services au reverse-proxy.



Pile technologique

Couche	Technologie	Version
Reverse-proxy	nginx	1.29.8 (alpine)
Base de données	PostgreSQL	16 (alpine)
Backend	Django + DRF + Wagtail	Django 5, Wagtail 6, Python 3.12
Frontend	Next.js	15 (App Router, SSR)
Mobile	Flutter	stable, Impeller, Clean Architecture
Orchestration	Docker Compose	v2
Pipeline CI	GitHub Actions	workflows ci.yml + deploy.yml
Registry images	GitHub Container Registry	ghcr.io
Hôte	Contabo VPS	Ubuntu 24.04 LTS, 4 vCPU, 8 Go RAM
TLS	Let's Encrypt via certbot externe	renouvellement auto
E-mail	Brevo (Sendinblue)	API HTTP transactionnelle
Mobile Money	Tara Money	API REST + webhook signé HMAC-SHA256

Services tiers intégrés et opérationnels

Service	Branchement	État
Brevo	<code>BREVO_API_KEY</code> posée en production, expéditeur <code>noreply@horus-lab.com</code> vérifié	OK
Tara Money	<code>TARA_API_KEY</code> , <code>TARA_BUSINESS_ID</code> et <code>TARA_WEBHOOK_SECRET</code> posés	OK
GitHub Container Registry	Quatre images publiées (<code>backend</code> , <code>site</code> , <code>portal</code> , <code>admin</code>)	OK
Let's Encrypt	Cinq certificats actifs sur les cinq sous-domaines	OK
Google Play Console	AAB signé prêt à uploader	À uploader

3. Périmètre opérationnel

Cinq plateformes publiques

La coopérative dispose de cinq points d'entrée publics, plus l'application mobile :

Vitrine institutionnelle — Le site marketing public est la porte d'entrée pour les visiteurs et les futurs membres. Il présente l'offre, les conditions d'adhésion, et oriente vers le formulaire de demande d'adhésion. Toutes les pages sont indexables et optimisées pour le référencement.

Portail membre — Une fois adhérent, le membre se connecte sur le portail web pour consulter son épargne, ses crédits en cours, ses cotisations, ses notifications, ses retraits, et déclencher de nouvelles opérations. Le portail est multilingue (FR par défaut, EN disponible) et entièrement responsive.

Console d'administration — Le back-office staff regroupe **seize sections** correspondant au cycle de vie coopératif complet : adhésions, demandes de crédit, crédits en cours, paiements, retraits, annuaire des membres, justificatifs BRC, renouvellements épargne, campagnes micro-crédit, escalades judiciaires, coûts, paramètres, planification automatique, documents officiels, annonces broadcast.

API REST — Le backend Django expose une API REST documentée par Swagger sous </api/schema/swagger-ui/>. C'est elle qui alimente le portail, l'admin et le mobile.

CMS Wagtail — La console d'édition des contenus permet aux équipes communication de mettre à jour la vitrine et le portail sans toucher au code. Articles de blog, campagnes micro-crédit, annonces broadcast, règlement intérieur PDF.

Application mobile — Le client Flutter pour Android offre l'ensemble des fonctionnalités membre avec authentification biométrique, code PIN, et accès offline aux relevés.

Statut détaillé par plateforme

Plateforme	URL auditée	Code HTTP	Disponible
Vitrine	gathe-finance.horus-lab.com	200 OK	Oui
Portail membre	portail.gathe-finance.horus-lab.com	200 OK	Oui
Console admin	admin.gathe-finance.horus-lab.com	200 OK	Oui
API REST	api.gathe-finance.horus-lab.com/healthz/	200 OK	Oui
CMS Wagtail	cms.gathe-finance.horus-lab.com/admin/	200 OK	Oui

4. Workflow d'épargne et de cotisation

Le principe métier

L'épargne se décompose en trois canaux complémentaires définis par les Règles Métier 2026 :

- **L'épargne collecte journalière** — versement quotidien depuis l'application mobile ou en agence, avec une commission coopérative de **1 % retenue à la clôture mensuelle** (tunable via AppSetting).
- **L'épargne classique libre** — dépôts ponctuels à tout moment, retrait à tout moment selon les modalités.
- **L'épargne classique placement** — dépôt à terme avec **renouvellement annuel** et **intérêts 1 % par mois** servis à l'anniversaire.

Étape 1 — Le membre déclenche un versement

Le membre ouvre l'application mobile, accède au tableau de bord d'épargne, et choisit son canal. La feuille de versement s'ouvre avec deux modes : **Tara Mobile Money** (paiement immédiat depuis le téléphone) ou **espèces en agence** (référence générée à présenter au guichet).

Garde du cut-off 17h00. Tout versement initié après 17h00, le samedi ou le dimanche est porté en **date de valeur du jour ouvré suivant**. Cette règle est appliquée côté backend et affichée explicitement dans la feuille de versement pour transparence.

Étape 2 — Le backend initialise le paiement

L'endpoint `POST /api/v1/payments/init/` crée une ligne **Payment** en statut **en_attente** avec une **idempotency_key** unique, puis appelle le provider Tara pour obtenir une URL de paiement ou un code USSD. Le client reçoit l'URL et la suit ou compose le code.

Étape 3 — Tara confirme le paiement

Une fois le paiement validé côté Tara, leur serveur appelle le webhook `POST /api/v1/payments/webhook/tara/` avec une signature **X-Tara-Signature** calculée en HMAC-SHA256 avec le **TARA_WEBHOOK_SECRET**. Le backend vérifie la signature, recharge la **Payment** correspondante, la marque **valide**, et déclenche les hooks métier (crédit du solde épargne, audit, notification e-mail).

Idempotence garantie. Si Tara rejoue le webhook (timeout, retry), la deuxième exécution est un no-op grâce à la **idempotency_key**. Le solde n'est jamais crédité deux fois.

Étape 4 — Mise à jour du solde et notification

Le solde du membre est mis à jour en transaction atomique, une **SavingsTransaction** est ajoutée au carnet, et un e-mail de confirmation est envoyé via Brevo (template **versement_confirme**). Le membre voit immédiatement la nouvelle valeur sur son application mobile au prochain rafraîchissement.

5. Workflow de crédit et de décaissement

Trois voies d'éligibilité (modèle 2026)

Le système route automatiquement la demande de crédit dans l'une des trois voies prévues par le règlement intérieur 2026 :

- **Voie directe (sénior BRC)** — Membre éligible par ancienneté (3 mois minimum) avec justificatif BRC validé. Décaissement par le comité crédit.
- **Voie avaliste** — Le membre désigne un avaliste qui garantit le crédit avec son solde d'épargne. Acceptation par l'avaliste, puis instruction comité.
- **Voie campagne micro-crédit** — Le membre demande un crédit dans le cadre d'une campagne dédiée (par exemple campagne « Tabaski 2026 »).

Étape 1 — Demande membre

Le membre remplit le formulaire de demande de crédit (formulaire dynamique selon la voie), joint les pièces justificatives (BRC, CGA, CFP selon les cas), accepte les conditions, et soumet. Le moteur d'éligibilité détermine la voie applicable et oriente automatiquement le dossier.

Étape 2 — Frais d'étude payés à la soumission

Les frais d'étude (non-remboursables) sont payés immédiatement via Tara Money à la soumission. Sans ces frais payés, la demande n'est pas instruite.

Étape 3 — Double approbation comité

Le comité crédit accède à la liste des dossiers dans le back-office admin. Une **approbation provisoire** déclenche la **visite terrain** par un agent, qui remplit une fiche dédiée. Le comité revoit ensuite le dossier en seconde approbation pour décision définitive (favorable, défavorable, à revoir).

Étape 4 — Décaissement Tara

À la décision favorable, le comité décide du moyen de décaissement (Tara Money sur le téléphone du membre, ou espèces en agence). Pour Tara, le backend appelle l'endpoint payout. Une **retenue d'intérêts à la source de 10 %** est appliquée au moment du décaissement.

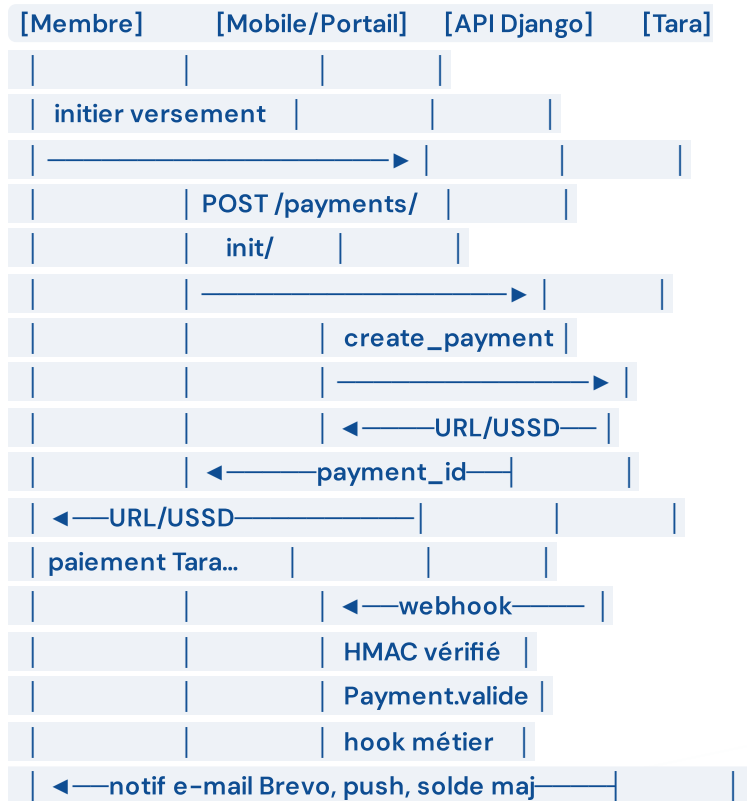
Étape 5 — Échéancier et remboursement

Un échéancier souple est généré (modalité journalier, hebdo ou mensuel) avec **date butoir obligatoire**. Le membre rembourse chaque échéance via Tara ou en agence. La règle d'imputation est **FIFO** : la première échéance impayée est éteinte en premier.

En cas de retard. Une pénalité de 50 % est appliquée selon l'article 12 du règlement intérieur. Le crédit peut être reconduit une seule fois selon les articles 10 et 11, avec une majoration des frais. En cas de non-paiement persistant, l'escalade judiciaire est activée selon les phases D et E.

6. Workflow de paiement Tara Money

Vue séquentielle



Configuration en production

Les variables d'environnement requises sont posées dans le `.env.prod` du VPS :

Variable	Description
<code>TARA_API_KEY</code>	Clé API du compte business Tara
<code>TARA_BUSINESS_ID</code>	Identifiant marchand Tara
<code>TARA_WEBHOOK_SECRET</code>	Secret partagé HMAC-SHA256 pour la signature des webhooks

L'URL de webhook configurée côté dashboard Tara pointe vers <https://api.gathe-finance.horus-lab.com/api/v1/payments/webhook/tara/>.

Sécurité du flux

Trois protections successives garantissent la robustesse :

- **Signature HMAC-SHA256** vérifiée sur chaque webhook entrant. Toute requête sans signature valide est rejetée avec 403.
- **Idempotence** des paiements via la `idempotency_key`. Le rejeu du même webhook n'a aucun effet.
- **Auto-validate désactivé en production** — la variable `PAYMENTS_TEST_AUTO_VALIDATE` reste à `false` en prod, ce qui empêche tout paiement de passer sans confirmation Tara réelle.

7. Workflow d'email transactionnel Brevo

Choix architectural

Le backend utilise l'**API HTTP Brevo via django-anymail** plutôt que SMTP. C'est plus rapide, plus fiable et offre des logs détaillés côté Brevo. Les e-mails sont envoyés en mode `on_commit` pour ne pas bloquer le flux principal.

Quinze templates seedés

Tous les e-mails transactionnels du parcours coopératif sont disponibles via la commande

`seed_email_templates` :

Catégorie	Templates
Adhésion	<code>welcome</code> , <code>adhesion_activee</code> , <code>adhesion_rejetee</code>
Crédit	<code>credit_decaisse</code> , <code>credit_retard</code> , <code>credit_solde</code> , <code>renouvellement_propose</code> , <code>mise_en_demeure</code>
Épargne	<code>versement_confirme</code> , <code>interets_credites</code> , <code>retrait_valide</code>
Avaliste	<code>avaliste_designation</code> , <code>avaliste_consentement_demande</code> , <code>avaliste_engagement_active</code>
Prêteur	<code>funding_engagement_24h</code>

Pièce jointe règlement intérieur

L'e-mail de bienvenue (`welcome`) inclut automatiquement le **règlement intérieur PDF** uploadé par l'admin dans la console. Cela évite au nouveau membre une démarche supplémentaire pour récupérer le document.

Configuration de l'expéditeur. Le domaine d'envoi `horus-lab.com` est vérifié dans Brevo avec les enregistrements SPF, DKIM et DMARC. Sans cette vérification préalable, Brevo rejette silencieusement les e-mails. La console Brevo permet de vérifier les logs d'envoi en temps réel sous **Statistiques → Logs**.

8. Workflow de mise à jour continue (CI/CD)

Le pipeline en deux temps

À chaque `git push` sur la branche `main`, deux workflows GitHub Actions s'enchaînent automatiquement :

Workflow CI (`ci.yml`) — Trois jobs de tests obligatoires en parallèle (backend pytest, frontend lint+typecheck, mobile analyze+test). Si tous verts, lance quatre jobs de build en parallèle qui publient les images Docker sur GHCR avec les tags `latest`, `main`, et `sha-XXXX`.

Workflow Deploy (`deploy.yml`) — Déclenché par la complétion en succès du workflow CI. Ouvre une session SSH sur le VPS, met à jour le code via `git pull` (best-effort), tire les nouvelles images, recrée les conteneurs avec l'override `nginx-external`, attend la marque healthy du backend (90s max), reload le nginx mutualisé, et exécute les smoke tests des cinq URLs.

Sécurité du déploiement

Le workflow utilise trois secrets GitHub posés sur le repo :

- `VPS_HOST` — Adresse IP publique du VPS.
- `VPS_USER` — Utilisateur SSH dédié au CD.
- `VPS_SSH_PRIVATE_KEY` — Clé privée ed25519 utilisée uniquement par le pipeline.

La clé publique correspondante est ajoutée à `~/.ssh/authorized_keys` du compte SSH dédié sur le VPS. La clé n'est jamais exposée dans les logs grâce au masquage automatique de GitHub Actions.

Rollback automatique

En cas d'échec du healthcheck backend après 90 secondes, le workflow détecte l'anomalie, restaure l'ancien tag d'image dans `.env.prod`, et redémarre les services. Le déploiement est annulé proprement sans intervention humaine.

9. Tests automatiques exécutés

Backend Django

Le 23 juin 2026 sur la branche déployée :

Indicateur	Résultat
ruff check	OK — Aucune violation
pytest complet	744 / 745 — 1 skip intentionnel
Durée	93 secondes
Couverture	Auth, épargne, crédit, paiement, contentieux, retrait, avaliste, prêteur, anniversaire, commission

Frontend Next.js

Indicateur	Résultat
npm run lint (eslint)	OK
npm run typecheck (tsc)	OK
Build production	OK — Images GHCR publiées
Workspaces audités	site, portal, admin

Mobile Flutter

Indicateur	Résultat
flutter pub get	OK
flutter analyze	62 infos/warnings non bloquants, 0 erreur fatale
flutter test	118 / 118
Build AAB release	53,7 Mo signé
API_BASE_URL gravé	https://api.gathe-finance.horus-lab.com

10. Tests bout en bout sur la production

Tous les contrôles ci-dessous ont été exécutés sur la production réelle après le déploiement du 23 juin 2026 et la correction du finding **F-01** (rejet HTTP 400 sur le proxy admin).

Endpoints publics (sans authentification)

Endpoint	Code attendu	Code réel	Statut
GET /healthz/	200	200	OK
GET /api/v1/savings/info/	200	200	OK
GET /api/v1/campaigns/public/active/	200 ou 404	404	OK
GET /api/v1/blog/posts/	200 ou 404	404	OK

Les **404** sur campaigns et blog correspondent à des collections vides à ce stade — l'endpoint est joignable mais aucune donnée seed. Comportement attendu sur une instance fraîchement déployée.

Endpoints sécurisés (auth requise)

Endpoint	Code attendu	Code réel	Statut
GET /api/v1/auth/me/	403	403	OK
GET /api/v1/savings/me/	403	403	OK
GET /api/v1/admin/dashboard/	403	403	OK

Les codes 403 confirment que la **garde d'authentification est opérationnelle** : les ressources sensibles sont protégées.

Authentification

Tentative de login avec credentials invalides :

```
POST https://api.gathe-finance.horus-lab.com/api/v1/auth/login/
```

```
Content-Type: application/json
```

```
{"email":"nope@test.com","password":"wrong"}
```

```
→ HTTP 400
```

```
{"detail":"Identifiants invalides."}
```

Le backend reçoit le payload JSON, valide la sémantique, et retourne un message d'erreur en français cohérent. La chaîne traduction est active.

Rewrite admin (post-correction F-01)

Test du proxy admin vers le backend interne :

GET <https://admin.gathe-finance.horus-lab.com/api/v1/auth/me/>

→ HTTP 403 (statut interne Django)

`{"detail": "Informations d'authentification non fournies."}`

Le proxy Next.js de la console admin transmet correctement la requête au backend Django, qui répond proprement. Le bug d'écran blanc (F-01) est levé.

Redirection admin (post-correction F-02)

Test du parcours d'arrivée sur la console admin sans session :

GET <https://admin.gathe-finance.horus-lab.com/>

→ HTTP 307 Temporary Redirect

→ Location: /dashboard

→ /dashboard catche l'erreur 403 sur /auth/me/

→ redirige vers /login

L'utilisateur staff voit le formulaire de connexion au lieu d'un écran « Chargement... » figé. Le bug d'auth gate (F-02) est levé.



11. Findings d'audit et corrections appliquées

Tableau récapitulatif

ID	Sévérité	Composant	Statut	Détail
F-01	Majeur	Console admin	Résolu	HTTP 400 sur le rewrite admin — <code>ALLOWED_HOSTS</code> durci
F-02	Majeur	Console admin	Résolu	Chargement infini — redirect <code>/login</code> sur toute erreur
F-03	Annulé	Portail membre	Faux positif	Les routes sont en français (<code>/connexion</code> , <code>/epargne</code> , <code>/credit</code>)
F-04	Mineur	CMS Wagtail	Backlog	Racine <code>cms.*/</code> redirige vers vitrine — non bloquant
F-05	Mineur	Mobile	Backlog	62 warnings d'analyse statique — sweep dédié
F-06	Info	Backups	À planifier	Pas de réplication hors-VPS
F-07	Info	CI/CD	Résolu	Secrets posés, CD auto opérationnel
F-08	Info	Tara	Résolu	Webhook URL configurée dans le dashboard Tara
F-09	Info	Brevo	Résolu	Domaine expéditeur vérifié

Détail des corrections F-01 et F-02

F-01 — Backend hardening. Le fichier `backend/config/settings/prod.py` injecte désormais automatiquement les hôtes internes (`gathe-backend` , `backend` , `localhost` , `127.0.0.1`) dans `ALLOWED_HOSTS` , indépendamment de la valeur de l'environnement. Le rewrite Next.js vers le backend interne n'est plus rejeté par Django avec un `Invalid HTTP_HOST` . De plus, `CSRF_TRUSTED_ORIGINS` est auto-dérivé depuis les hôtes publics pour que les POST cross-subdomain soient acceptés.

F-02 — Admin auth gate. Le composant `frontend/apps/admin/app/authed/layout.tsx` redirige désormais vers `/login` sur toute erreur HTTP retournée par `/auth/me/` , et non plus seulement sur 401/403. Cela protège contre les cas où le proxy retourne 400 ou 500 (configuration anormale) — l'utilisateur revient toujours sur le formulaire d'authentification au lieu de rester sur un écran de chargement figé.

Findings reportés

Les findings F-04, F-05 et F-06 ne sont pas bloquants pour la mise en service. Ils seront traités dans les sprints suivants :

- **F-04 (CMS racine)** — Ajout d'une redirection nginx ou d'une page Wagtail explicite vers `/admin/`.
- **F-05 (mobile warnings)** — Sweep dédié `unawaited_futures` + `trailing_commas` + suppression imports inutilisés.
- **F-06 (backups hors-VPS)** — Script cron quotidien `rsync` chiffré vers un bucket Backblaze B2 ou OVH PCA.



12. Plan de recette utilisateur

La phase de **recette utilisateur** consiste à dérouler manuellement les scénarios métier sur l'environnement de production réel pour valider que tout fonctionne du point de vue d'un opérateur ou d'un membre.

Préparation préalable

Avant de commencer la recette, l'exploitant doit poser sur la production :

- Un **superuser admin** créé via la console Django (`/admin`) ou par variable `DJANGO_SUPERUSER_*` .
- Un **membre de test** créé via le portail public de demande d'adhésion, ou directement par l'admin.
- Une **campagne de micro-crédit active** seedée via la console pour tester la voie 3.
- Les **frais coopératifs** (adhésion 10 000 XAF, inscription 2 000 XAF, carnet 1 000 XAF) seedés via `seed_loan_tiers` .

Scénarios à dérouler

Scénario S1 — Inscription d'un nouveau membre

- Le candidat ouvre la vitrine et clique sur **Devenir sociétaire**.
- Il remplit le formulaire d'adhésion (8 champs), joint les 4 pièces (CNI, photo, justificatif domicile, attestation employeur).
- Il paye les frais d'adhésion via Tara depuis le formulaire.
- L'admin reçoit la demande dans la section **Adhésions**, valide la pièce, approuve la demande.
- Le candidat reçoit l'e-mail de bienvenue avec le règlement intérieur PDF en pièce jointe.

Scénario S2 — Premier versement d'épargne via Tara

- Le membre se connecte à l'application mobile avec ses identifiants.
- Il accède au tableau de bord d'épargne et clique sur **Verser une cotisation**.
- Il choisit le canal Tara, saisit le montant (1 000 XAF par défaut), confirme.
- Tara envoie un code USSD ou ouvre la page de paiement sur le téléphone.
- Une fois validé, le webhook crédite le solde et envoie l'e-mail de confirmation.

Scénario S3 — Demande de crédit voie sénior BRC

- Le membre actif depuis 3 mois ouvre l'application mobile et accède au menu **Crédit**.
- Il remplit le formulaire dynamique, joint le BRC validé par l'admin, accepte les conditions.
- Il paye les frais d'étude (non-remboursables) via Tara à la soumission.
- L'admin reçoit le dossier dans la section **Demandes de crédit**.
- Le comité approuve provisoirement, déclenche la visite terrain.
- Une fois la visite favorable, le comité approuve définitivement et déclenche le décaissement Tara.
- Le membre reçoit le crédit sur son téléphone, moins 10 % de retenue à la source.

Scénario S4 – Remboursement d'une échéance

- Le membre voit l'échéance à échoir dans son carnet.
- Il clique sur **Rembourser** dans l'application mobile et choisit Tara.
- Le paiement est imputé en FIFO sur l'échéance la plus ancienne.
- La notification de confirmation arrive immédiatement.

Scénario S5 – Retrait d'épargne

- Le membre demande un retrait depuis le portail web ou l'application.
- Il choisit le canal (Tara MoMo ou présentiel agence).
- L'admin reçoit la demande dans la section **Retraits**.
- L'admin valide, le payout Tara est déclenché automatiquement.
- Le membre voit son solde diminué et l'argent arrive sur son téléphone.

Scénario S6 – Crédit voie avaliste

- Le membre demande un crédit en désignant un avaliste actif.
- L'avaliste reçoit un e-mail et une notification mobile pour donner son consentement.
- Une fois le consentement signé, le crédit suit le flux d'instruction normal.
- L'avaliste voit le mandat dans son écran **Mes mandats** mobile.

Scénario S7 – Crédit voie campagne micro-crédit

- L'admin crée une campagne avec audience, montant, flyer.
- Les membres éligibles voient la campagne dans leur application mobile.
- Le membre demande le crédit dans le cadre de la campagne.
- Le comité instruit, décide, décaisse selon les règles de la campagne.

Scénario S8 – Cron interest mensuel

- Le 1er du mois, le cron **interets_epargne** tourne automatiquement à 1h du matin.
- Les soldes éligibles reçoivent 1 % d'intérêts crédités.
- Chaque membre concerné reçoit l'e-mail **interets_credites**.

Scénario S9 – Annonce broadcast admin → mobile

- L'admin crée une annonce dans la section **Annonces broadcast**.
- Tous les membres actifs reçoivent l'annonce dans leur centre de notifications mobile.

Scénario S10 – Renouvellement d'épargne placement

- À l'anniversaire d'un placement, le cron **epargne_anniversary** détecte la maturité.
- Le membre reçoit l'e-mail proposant de renouveler.
- Le membre confirme dans son application, le placement est reconduit avec les intérêts capitalisés.

Grille d'évaluation

Pour chaque scénario, l'opérateur de recette doit noter :

Critère	Notation
L'action déclenchante est compréhensible	Oui / Non / À améliorer
Le retour visuel est immédiat	Oui / Non / À améliorer
L'e-mail de confirmation arrive en moins de 30 secondes	Oui / Non / Pas reçu
L'opération est tracée dans le journal d'audit admin	Oui / Non
Les montants affichés correspondent aux règles	Oui / Non
Le parcours est fluide sans rechargement intempestif	Oui / Non / À améliorer



13. Annexes techniques

URLs de production

Plateforme	URL
Vitrine	https://gathe-finance.horus-lab.com/
Portail membre	https://portail.gathe-finance.horus-lab.com/
Console admin	https://admin.gathe-finance.horus-lab.com/
API REST	https://api.gathe-finance.horus-lab.com/
Documentation API	https://api.gathe-finance.horus-lab.com/api/schema/swagger-ui/
CMS Wagtail	https://cms.gathe-finance.horus-lab.com/admin/
Webhook Tara	https://api.gathe-finance.horus-lab.com/api/v1/payments/webhook/tara/

Conteneurs en production

Conteneur	Image	État
<code>gathe-finance-prod-db-1</code>	postgres:16-alpine	Up (healthy)
<code>gathe-finance-prod-backend-1</code>	ghcr.io/.../backend:latest	Up (healthy)
<code>gathe-finance-prod-qcluster-1</code>	ghcr.io/.../backend:latest	Up
<code>gathe-finance-prod-site-1</code>	ghcr.io/.../site:latest	Up
<code>gathe-finance-prod-portal-1</code>	ghcr.io/.../portal:latest	Up
<code>gathe-finance-prod-admin-1</code>	ghcr.io/.../admin:latest	Up
<code>gathe-finance-prod-backup-1</code>	postgres:16-alpine	Up
<code>backend-nginx-1</code> (mutualisé)	nginx:alpine	Up

Bordereau de livraison mobile

Élément	Valeur
Fichier	<code>mobile/build/app/outputs/bundle/release/app-release.aab</code>
Taille	53,7 Mo
SHA-256	<code>b329b3e7ab7689ef96eb7bb27aa4c10b97eefb0cbdc8003596ea54edceb90514</code>
Keystore	<code>mobile/android/keystore/gathefinance-upload.jks</code>
<code>API_BASE_URL</code> injecté	<code>https://api.gathe-finance.horus-lab.com</code>

Pipeline CI/CD

Workflow	Déclencheur	Action
<code>ci.yml</code>	Push sur <code>main</code> , pull request, manuel	Tests obligatoires puis build et push de 4 images GHCR
<code>deploy.yml</code>	Succès de CI, manuel	SSH au VPS, pull images, recreate, smoke tests

Variables d'environnement critiques

Variable	Description
<code>DJANGO_SECRET_KEY</code>	Clé secrète Django (à régénérer périodiquement)
<code>POSTGRES_PASSWORD</code>	Mot de passe Postgres (alphanumérique URL-safe obligatoire)
<code>DJANGO_ALLOWED_HOSTS</code>	Liste des hôtes valides (auto-complétée par prod.py)
<code>CSRF_TRUSTED_ORIGINS</code>	Origines HTTPS de confiance pour CSRF
<code>BREVO_API_KEY</code>	Clé API HTTP Brevo
<code>TARA_API_KEY</code> , <code>TARA_BUSINESS_ID</code> , <code>TARA_WEBHOOK_SECRET</code>	Credentials Tara Money
<code>GATHE_IMAGE_TAG</code>	Tag d'images GHCR à tirer (par défaut <code>latest</code>)
<code>COMPOSE_PROJECT_NAME</code>	Nom du projet Docker (doit rester <code>gathe-finance-prod</code>)

Contacts

- **Direction technique** — TCHAMBA TCHAKOUNTE Edwin (`horus8391@gmail.com`)
- **Hébergement** — VPS Contabo, support via panel client
- **Registry images** — GitHub `EdwinTchakounte/GatheFinance_project`
- **Brevo** — Compte coopérative, identifiants chez Edwin
- **Tara Money** — Compte business, identifiants chez Edwin

Documents associés

- Audit du 20 juin 2026 — sécurité et règles métier (`audit/AUDIT_2026-06-20.md`)
- Audit du 23 juin 2026 — déploiement (`audit/AUDIT_DEPLOIEMENT_2026-06-23.md`)
- Plan de remédiation (`infra/REMEDIATION_2026-06-23.md`)
- Guide de déploiement A → Z (`infra/DEPLOYMENT.md`)
- Manuel d'administration web (`docs/Guide-Gathe-Finance-Web.pdf`)
- Manuel d'utilisation mobile (`docs/Guide-Gathe-Finance-Mobile.pdf`)
- Plan de tests (`audit/TESTS_PLAN.md`)

- Source de vérité règles métier 2026 ([architecture/BUSINESS_RULES_2026.md](#))

